

Agentic Healthcare Assistant

Project overview: flow, services, AI/RAG design, and security model.

1. What it is

A Streamlit application for patients, doctors and attendants. Patients ask medical questions, view their own or a relative's records, and book appointments. Doctors and attendants review patient summaries, upload reference documents, and manage records. An LLM planner converts each request into small steps (goals), which the application validates and executes in dependency order.

2. Request flow

```
User request (Streamlit)
|
Planner: OpenAI returns a strict JSON plan --> plan validation
|
PlanExecution: goals run in dependency order
  patient_lookup --> history_retrieval -----+
      |                                           |
      +--> specialist_discovery --> appointment -----+
  medical_question (RAG over reference documents) -----+--> final_summary
  medical_search (WHO + PubMed) -----+                |
                                           Answer, citations,
                                           notices
```

1. **Planning** (`src/agents/planner.py`): OpenAI returns a strict JSON plan. Allowed goals: `patient_lookup`, `history_retrieval`, `specialist_discovery`, `appointment`, `medical_question`, `medical_search`, `conversation_recall`, `final_summary`. The plan is validated before running.
2. **Execution** (`src/agents/plan_execution.py`): goals run in dependency order; a failed goal blocks only its dependents. `final_summary` always runs and reports failures.
3. **Final answer**: combines history summary, RAG answer, publication findings, OpenAI treatment suggestions and booking result, plus coverage and search notices.

3. Services and endpoints

Purpose	Service	Where
Chat, planning, summaries	OpenAI POST <code>/v1/chat/completions</code> (default model <code>gpt-4o-mini</code> , <code>PLANNER_MODEL</code>)	<code>src/llm/planning_client.py</code>
Embeddings	OpenAI POST <code>/v1/embeddings</code>	<code>src/vector_store/patient_summaries.py</code>
Medical literature	PubMed via NCBI E-utilities (<code>esearch.fcgi</code> , <code>efetch.fcgi</code>)	<code>src/tools/medical_search.py</code>

Medical literature	WHO publications OData API (/api/hubs/publications)	src/tools/medical_search.py
Scheduling	Private HTTP API: POST /specialists , /slots , /bookings (bearer-token auth, SQLite-backed)	src/scheduling/api.py
Storage	SQLite: patients, history, prescriptions, alerts, appointments, conversations, events, summary vectors	data/healthcare.db
Deployment	Streamlit on Railway; health check /_stcore/health	railway.json

4. AI, RAG and embeddings

Embeddings

- Reference documents: text-embedding-ada-002 (EMBEDDING_MODEL).
- Patient summaries: text-embedding-3-small (PATIENT_SUMMARY_EMBEDDING_MODEL).
- Vectors are L2-normalised, so inner product behaves as cosine similarity.

Vector stores (FAISS)

- **Reference index** (FAISSStore): one shared index, persisted atomically in SQLite as serialized FAISS bytes plus JSON metadata (no pickle). Records the embedding model and rebuilds if it changes.
- **Patient summaries** (PatientSummaryStore): one IndexFlatIP per patient in table patient_summary_vectors , tied to a fingerprint of the source records; a changed record makes the index stale and forces a rebuild.
- A ChromaDBStore exists but is not part of the main flow.

Document ingestion

PDF, then page extraction, then chunks (1000 characters, 200 overlap; CHUNK_SIZE , CHUNK_OVERLAP), then embeddings, then the FAISS index. Only doctors and attendants can upload.

RAG

- Retrieves the top 5 chunks from the reference index and answers only from that context; returns a fixed "no reference answer" message when nothing is retrieved.
- Patient questions also receive the structured patient snapshot, the top 5 summary excerpts, and live publication evidence.
- Follow-up questions are rewritten into standalone questions using chat history.
- With no reference index, the app uses live search, then a general OpenAI answer.

Live medical search (grounded evidence)

- An LLM extracts a de-identified topic from the question and recorded conditions; only that topic leaves the app (no names or IDs).

- Last 730 days; retracted papers excluded; responses size-limited; redirects blocked; PubMed calls rate limited.
- The summary uses a JSON schema permitting only retrieved source IDs; the app attaches verified links itself.
- When publications are found, OpenAI treatment suggestions are appended (labelled as general knowledge). When none are found, only the OpenAI suggestions are shown.

Doctor treatment plan

The doctor enters a condition. The app combines the patient's stored history, summary vectors and live search findings, and OpenAI drafts a plan. The doctor makes the final decision.

Other concepts

- **Structured output:** strict JSON schemas for plans, summaries and findings, followed by validation.
- **Source-attributed summarization:** statements cite record IDs (`history:<id>`, etc.); completeness is checked (every prescription, every active alert).
- **Prompt-injection defence:** patient, publication and user text are treated as untrusted data.
- **Conversation memory:** turns saved per requester and subject, recalled only with access.
- **Evaluation:** cases in `evaluation/` and `src/evaluation/`; a performance dashboard reports results.

5. Access and privacy

- Roles: patient, doctor, attendant (login records in SQLite).
- Family access via dependents and permission grants; appointment requests for a relative are booked under the logged-in patient without that check.
- Staff see only authorised patients; access and record freshness are re-checked before and after generation.
- Bookings use idempotency keys to avoid double-booking; an event log records goal starts, completions and bookings.

6. Project layout

Path	Content
<code>app/</code>	Streamlit views (doctor notes, document upload, evaluation, performance, live appointments)
<code>src/agents/</code>	Planner, plan and goal execution, conversation flow
<code>src/llm/</code>	OpenAI client, prompts, summary renderers
<code>src/tools/</code>	Live medical search
<code>src/vector_store/</code> , <code>src/embeddings/</code> , <code>src/data_processing/</code>	RAG pipeline
<code>src/repositories/</code> , <code>src/scheduling/</code>	Data access and scheduling API
<code>scripts/</code>	Seeding, ingestion, scenario runs, model evaluation

tests/	Unit tests (pytest)
--------	---------------------

7. Running

- App: `python -m streamlit run app/streamlit_app.py`; requires `OPENAI_API_KEY` in `.env`.
- Scheduling API: `python -m src.scheduling.api --port 8765`; needs `SCHEDULE_API_TOKEN` (24+ characters) and `SCHEDULE_API_URL`.
- Optional: `NCBI_EMAIL`, `NCBI_API_KEY`, `MEDICAL_SEARCH_DAYS`.